

CAMPUS CONNECT

College Placement & Internship Management System

FINAL PROJECT REPORT

Project Type: Website Application

Technology Track: Java Website Application

Academic Year: 2026-27

Guide: Er. Ram Babu Buri

Team Leader: Vinod Yadav

Team Members: Ritika Yadav, Shalini Priya, Rakshit Sharma, Akshara Prashar

CERTIFICATE

This is to certify that the project titled "Campus Connect - College Placement & Internship Management System" has been developed as an academic project during the academic year 2026-27 under the guidance of Er. Ram Babu Buri. The project demonstrates a Java-based web application for centralized placement and internship management.

Signature: _____ Date: _____

DECLARATION

We declare that this project report describes the development, implementation, testing and documentation work carried out by the project team. The report is prepared for academic evaluation. Any official approval, signature, date or institutional seal should be completed by the college/guide before submission.

Signature: _____ Date: _____

ACKNOWLEDGEMENT

We sincerely thank Er. Ram Babu Buri for guidance and support during the project. We also thank the department and college for providing the academic environment and resources required for development, testing and documentation. We appreciate the contribution of all team members in completing the project.

Signature: _____ Date: _____

ABSTRACT

Campus Connect is a web-based College Placement and Internship Management System developed to provide a centralized platform for managing college placement and internship activities. The application provides separate role-based functionality for Students, Recruiters and Administrators.

Students can securely login, manage profile information, view available placement and internship opportunities, apply for suitable jobs, track application status, and view scheduled interviews and interview results. Recruiters can manage job opportunities, review student applications, update application status, shortlist or reject candidates, select candidates, and schedule interviews. Administrators can monitor the system through dashboards and database-driven reports.

The application uses Java 21, JSP, Servlets, JDBC, MySQL, Bootstrap 5, Maven and Apache Tomcat 9. The system follows MVC architecture and includes session management, role-based authorization, BCrypt password hashing, input validation and environment-based database configuration.

Keywords: Campus Connect, Placement Management, Internship Management, Java, JSP, Servlets, MySQL, JDBC, MVC

TABLE OF CONTENTS

1. Introduction
2. Problem Statement
3. Objectives
4. Existing System
5. Proposed System
6. Scope
7. User Roles
8. Technology Stack
9. System Requirements
10. System Architecture
11. Functional Requirements
12. Modules
13. Database Design
14. UML Design
15. UI Design
16. Security Implementation
17. Application Workflow
18. Implementation
19. Testing
20. Results and Observations
21. GitHub and Version Control
22. Development Progress
23. Future Enhancements
24. Limitations
25. Conclusion

26. References

Appendix

1. INTRODUCTION

1.1 Background

Placement and internship activities require coordination among students, recruiters and college administrators. A centralized system can reduce scattered records, simplify application tracking and make interview information easier to manage.

1.2 Purpose

Campus Connect provides a common web platform where students can discover and apply for opportunities, recruiters can manage jobs and applications, and administrators can monitor the overall placement workflow.

2. PROBLEM STATEMENT

2.1

Manual placement processes may involve spreadsheets, forms, messages and separate records. This can make student records, applications, interview schedules and placement statistics difficult to organize and track.

3. OBJECTIVES

3.1

The main objectives are to centralize placement activities, provide secure role-based access, enable student applications, support recruiter workflow, manage interviews and provide administrative statistics.

4. EXISTING SYSTEM

4.1

A conventional workflow may depend on separate tools for student data, job sharing, applications, interview coordination and reporting. Such separation can increase manual effort and make status tracking less transparent.

5. PROPOSED SYSTEM

5.1

Campus Connect integrates authentication, student profiles, job listings, applications, application status, interview management and reporting into one database-driven web application.

6. SCOPE OF THE PROJECT

- Students: profile management, job/internship viewing, applications, application status and interviews.
- Recruiters: company profile, job posting, application review, status updates, interview scheduling and result updates.
- Administrators: dashboard, monitoring of students/recruiters/companies/jobs/applications and reports.

7. USER ROLES

Role	Primary Responsibilities
Student	Profile, available jobs, applications, application status, interviews
Recruiter	Company, jobs, applications, candidate status, interviews
Administrator	System monitoring, management views, statistics and reports

8. TECHNOLOGY STACK

Layer	Technology
Frontend / View	JSP, HTML5, CSS3, JavaScript, Bootstrap 5
Backend / Controller	Java 21, Servlets
Database Access	JDBC
Database	MySQL
Server	Apache Tomcat 9
Build	Apache Maven
IDE / Version Control	Eclipse, Git, GitHub

9. SYSTEM REQUIREMENTS

9.1 Hardware

- Modern laptop or desktop computer
- At least 4 GB RAM recommended
- Sufficient storage for Java, Maven, Eclipse, Tomcat and MySQL
- Internet access for GitHub and project resources

9.2 Software

- Java Development Kit 21
- Eclipse IDE for Enterprise Java and Web Developers
- Apache Tomcat 9
- MySQL
- Apache Maven
- Git and GitHub
- Web browser

10. SYSTEM ARCHITECTURE

Campus Connect follows the Model-View-Controller architecture. JSP pages form the View layer, Java Servlets act as Controllers, DAO/model classes manage application data and database access, and MySQL stores persistent data.

```
Browser -> JSP/View -> Servlet/Controller -> DAO/JDBC -> MySQL
MySQL -> DAO/JDBC -> Servlet/Controller -> JSP/View -> Browser
```

11. FUNCTIONAL REQUIREMENTS

- User login with email and password.
- Role-based access for STUDENT, RECRUITER and ADMIN.
- Student profile management.
- View placement and internship opportunities.
- Apply for jobs and prevent duplicate applications.
- Track application statuses: APPLIED, SHORTLISTED, REJECTED, SELECTED.
- Recruiter job management and application review.
- Interview scheduling with date, time, mode, meeting link or venue.
- Interview result update: PENDING, PASSED, FAILED.
- Administrator dashboard and reports.
- Logout and session invalidation.

12. SYSTEM MODULES

12.1 Authentication Module

- Login
- Session creation
- Role identification
- Protected access
- Logout

12.2 Student Module

- Student dashboard
- Profile
- Available jobs
- Applications
- Interviews

12.3 Recruiter Module

- Recruiter dashboard
- Job management
- Application management
- Candidate status
- Interview management

12.4 Administrator Module

- Dashboard statistics
- Student/recruiter/company/job/application monitoring
- Reports

12.5 Reporting Module

- Overall statistics
- Application status distribution
- Job type distribution
- System summary

13. DATABASE DESIGN

The application uses a MySQL database named `campus_connect`. The database contains the following main tables and foreign-key relationships.

Table	Purpose
users	Authentication and basic user information
students	Student profile and academic details
companies	Recruiter/company information
jobs	Placement and internship opportunities
applications	Student applications and application status

interviews	Interview schedule and result information
notifications	User-specific notification records

Key relationships include Company -> Jobs, Student -> Applications, Job -> Applications, Application -> Interview and User -> Notifications. The students and companies tables reference users through user_id.

14. UML DESIGN

14.1 Use Case Diagram

The Use Case Diagram shows the interaction of Student, Recruiter and Administrator actors with the main system functions.

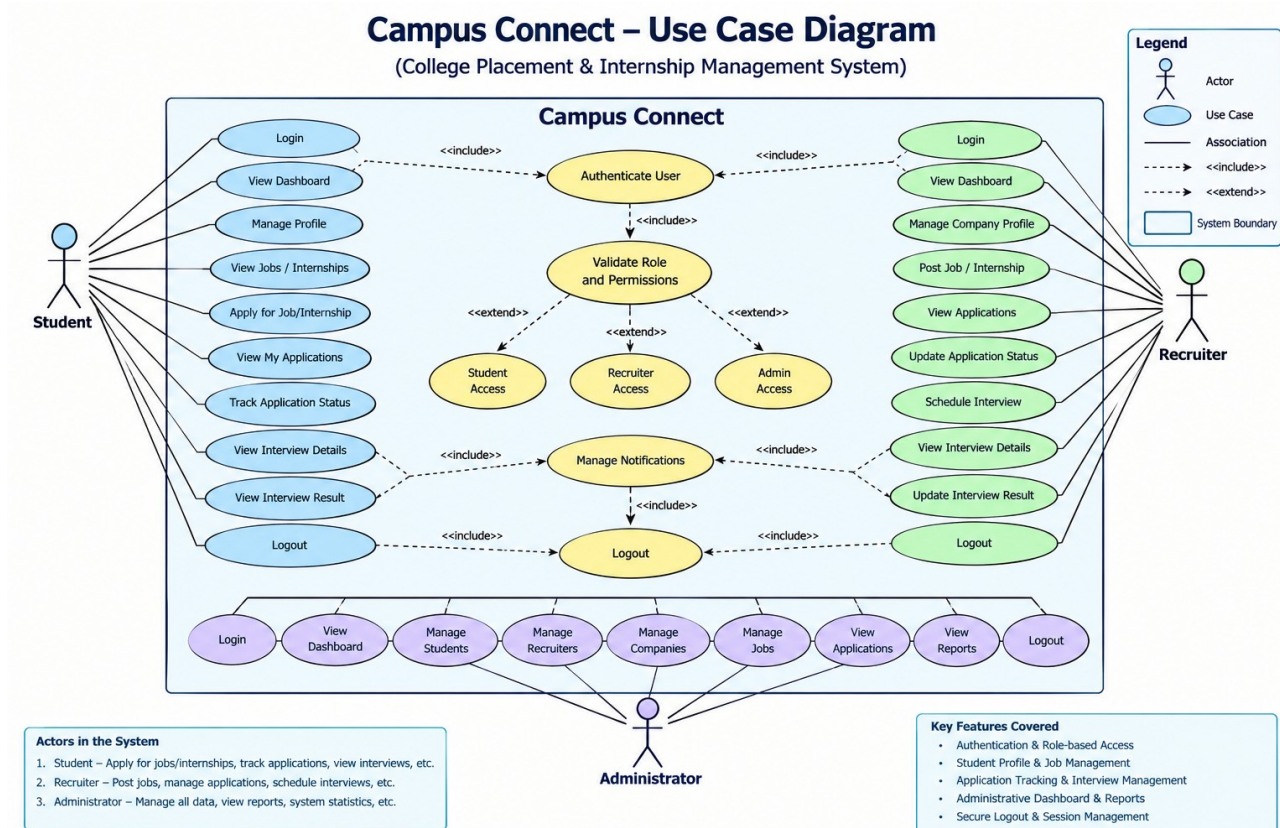


Figure 1 - Campus Connect Use Case Diagram

14.2 Class Diagram

The Class Diagram represents the principal domain classes and their relationships based on the implemented campus_connect database schema. Admin access is represented through the ADMIN role in the users entity.

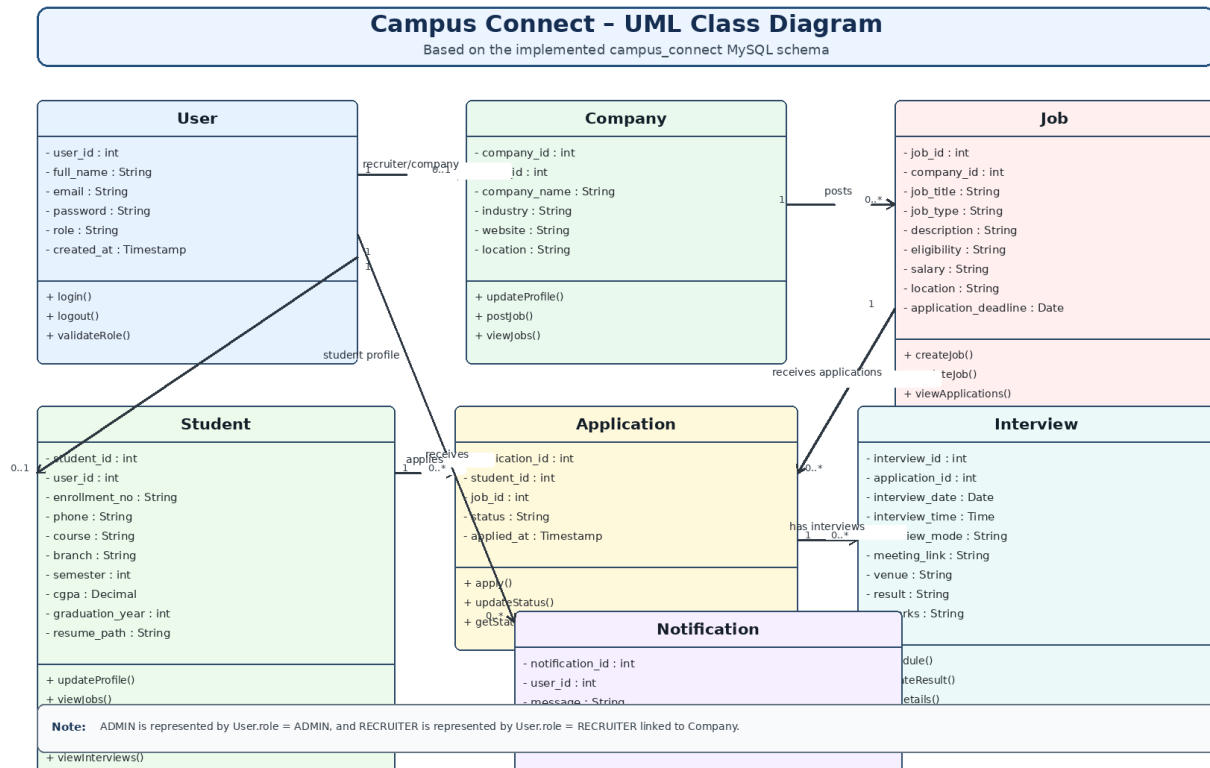


Figure 2 - Campus Connect Class Diagram

14.3 ER Diagram

The ER Diagram represents the implemented MySQL entities and their relationships.

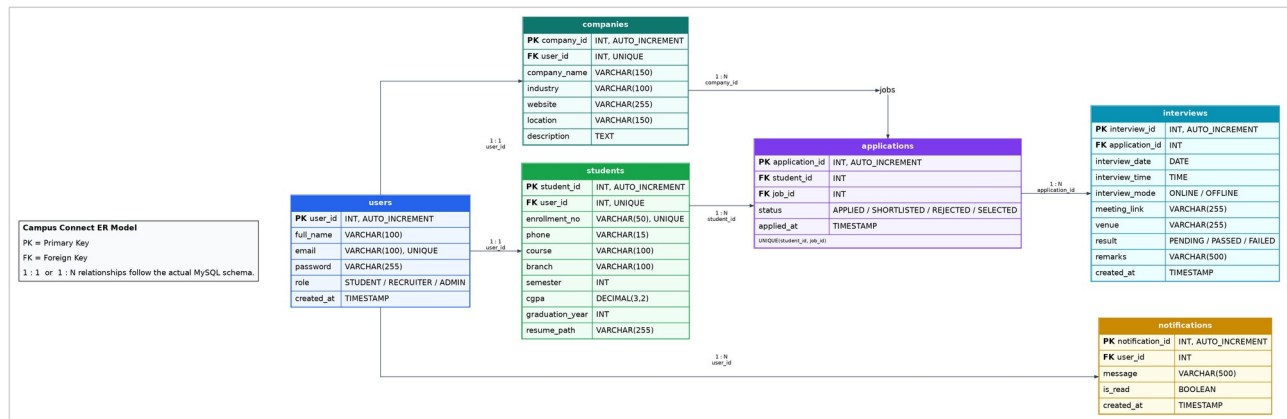


Figure 3 - Campus Connect ER Diagram

15. UI DESIGN

- Login page for all roles.
- Student dashboard with profile, jobs, applications and interviews.
- Recruiter dashboard with jobs, applications and interviews.
- Admin dashboard with system statistics and management views.
- Reports page with application status and job type statistics.
- Responsive layout using Bootstrap 5.

Detailed UI planning is maintained in docs/UI_Mockups.md.

16. SECURITY IMPLEMENTATION

- BCrypt hashing is used for password storage.
- Sessions are used to maintain authenticated users.
- Protected pages validate the session and user role.
- JDBC PreparedStatement is used for database operations.
- Database credentials are supplied through environment variables rather than hard-coded passwords.
- Input and request validation is applied to important operations.

17. APPLICATION WORKFLOW

17.1 Student Workflow

Login -> Dashboard -> View Jobs -> Apply -> My Applications -> Interview -> Result

17.2 Recruiter Workflow

Login -> Dashboard -> Post Job -> View Applications -> Shortlist/Reject/Select -> Schedule Interview -> Update Result

17.3 Administrator Workflow

Login -> Dashboard -> Students -> Recruiters -> Companies -> Jobs -> Applications -> Reports

18. IMPLEMENTATION

The project is implemented as a Maven-based Java web application. Java 21 is used for compilation, Apache Tomcat 9 hosts the application, and MySQL stores persistent data. JSP pages provide the view layer, Servlets handle requests and DAO classes encapsulate JDBC operations.

18.1 Maven

Manages dependencies, build configuration and WAR packaging.

18.2 JSP and Servlets

JSP provides page presentation and Servlets handle HTTP request processing.

18.3 JDBC

Provides database connectivity and execution of prepared SQL statements.

18.4 Tomcat

Provides the Servlet/JSP container required to run the web application.

19. TESTING

Functional and integration testing were performed across Student, Recruiter and Administrator modules. The complete placement workflow was also tested from application through interview result.

Area	Result
Authentication	PASS
Student Module	PASS
Recruiter Module	PASS
Admin Module	PASS
Applications	PASS
Interviews	PASS
Reports	PASS
Database Integration	PASS
Role-Based Access	PASS
Session Management	PASS

End-to-End Workflow	PASS
---------------------	------

19.1 Tested Student Functions

- Login
- Dashboard
- Profile
- Available jobs
- Job application
- Duplicate application prevention
- My applications
- Application status
- My interviews
- Interview result
- Logout

19.2 Tested Recruiter Functions

- Login
- Dashboard
- Job management
- Application listing
- Status updates
- Interview scheduling
- Interview listing
- Interview result update
- Logout

19.3 Tested Administrator Functions

- Login
- Dashboard
- Student, recruiter, company, job and application pages
- Reports
- Logout

20. RESULTS AND OBSERVATIONS

During final testing, the administrator reports page displayed the following current database values:

Metric	Observed Value
Total Students	1
Total Recruiters	1
Total Companies	1
Total Jobs	3
Total Applications	2
APPLIED	0

SHORTLISTED	2
SELECTED	0
REJECTED	0
PLACEMENT	1
INTERNSHIP	2

The complete end-to-end workflow from student application to recruiter shortlisting, interview scheduling, interview result update and student result viewing was verified.

21. GITHUB AND VERSION CONTROL

Git is used for version control and GitHub is used for repository hosting, backup and development history.

Repository: <https://github.com/vinodyadav895880-dotcom/campus-connect>

- Source code is maintained in the repository.
- The clean database schema is stored at database/campus_connect_schema.sql.
- Project documents and diagrams are stored under docs/.
- The repository contains .gitignore rules for target files, Eclipse metadata, logs and local configuration.

22. DEVELOPMENT PROGRESS

Stage	Major Work	Status
Planning	Team formation, guide, project idea	Completed
Requirements	SRS	Completed
Design	Use Case, Class, ER diagrams	Completed
Database/UI	Schema and UI documentation	Completed
Implementation	Student, Recruiter, Interview, Admin modules	Completed
Testing	Functional, security and integration testing	Completed
Documentation	Report and supporting documents	In progress
Presentation	PPT and demo preparation	Pending

Detailed progress tracking is maintained in docs/Weekly_Progress_Tracking.md. Any college-mandated signatures, meeting confirmations or daily evidence should be completed with actual records before submission.

23. FUTURE ENHANCEMENTS

- Email notifications
- Advanced job search and filtering
- Resume document management
- Automated eligibility checking

- Online aptitude tests
- Real-time notifications
- Mobile application
- Cloud deployment
- PDF/Excel report export
- Advanced placement analytics

24. LIMITATIONS

- The current application is primarily designed for college-level placement and internship workflows.
- Advanced notification services are not part of the current core implementation.
- Cloud deployment and mobile application are future enhancements.
- Automated eligibility analysis is not currently part of the core workflow.

25. CONCLUSION

Campus Connect provides a centralized web-based solution for college placement and internship management. The system integrates students, recruiters and administrators through role-based access and covers job discovery, applications, application status, interview scheduling, interview results and administrative reporting. The application demonstrates practical integration of Java web technologies, MySQL, MVC architecture, security controls and version control.

26. REFERENCES

- Java documentation
- Java Servlet API documentation
- JSP documentation
- MySQL documentation
- Apache Tomcat documentation
- Apache Maven documentation
- Bootstrap documentation
- Git documentation
- GitHub documentation
- Eclipse documentation

APPENDIX - PROJECT FILES

```
database/campus_connect_schema.sql
docs/Abstract.md
docs/Campus_Connect_SRS.md
docs/Use_Case_Diagram.png
docs/Class_Diagram.png
docs/ER_Diagram.png
docs/UI_Mockups.md
```

docs/Testing_Documentation.md
docs/Weekly_Progress_Tracking.md
docs/Campus_Connect_Final_Report.md
docs/Project_Run_Guide.md